# MIMO DETECTION ALGORITHMS

Python and C99 Simulation Study

| **Six receiver algorithms** |
| --- |
| Maximum Likelihood • Zero-Forcing • MMSE |
| ZF-SIC • MMSE-SIC • Lattice-Reduction-Aided |

**Prepared by**

PRANTHI (123EC0048)

CHETHAN (123EC0049)

Project report • Simulation methodology • Algorithm theory • Results and discussion

# Abstract

This project implements and studies six receiver-side detection algorithms for a spatial-multiplexing Multiple-Input Multiple-Output (MIMO) communication system. Standalone Python 3 and C99 programs simulate QPSK and 16-QAM transmission over AWGN, Rayleigh and Rician channel models. The receiver estimates transmitted symbols from the received signal and channel matrix. Performance is evaluated using Bit Error Rate (BER) over a Signal-to-Noise Ratio (SNR) sweep. The report explains the operating principle, motivation, practical use, distinguishing feature and computational trade-off of each detector.

## Project objectives

- Understand the baseband MIMO input-output model.
- Implement linear, successive-cancellation, exhaustive-search and lattice-reduction-aided detectors.
- Compare BER behaviour under a common test configuration.
- Relate detection performance to complexity and practical receiver constraints.

## Scope and interpretation

The programs are console-based educational simulations. They do not implement a complete cellular or WLAN physical layer. The checked-in BER values are finite Monte-Carlo estimates for one selected configuration; they are not theoretical guarantees or universal rankings.

## Contents

1. MIMO system model and simulation setup
2. Detection algorithms and method selection
3. Implementation and execution guide
4. BER results and discussion
5. Limitations and future improvements

# 1. MIMO system model

In a narrowband flat-fading spatial-multiplexing system, multiple transmit antennas send independent complex-valued symbols during the same channel use. The receive antennas observe a linear combination of these streams, modified by the channel and additive noise.

$$y = Hx + n$$

Here, x is the Nt × 1 transmitted symbol vector, y is the Nr × 1 received vector, H is the Nr × Nt channel matrix, and n is the additive complex Gaussian noise vector. The detector knows or is provided with H and estimates x from y.

## Key parameters

| Parameter | Meaning | Project options |
|---|---|---|
| Nt | Number of transmit antennas / spatial streams | 1–4 |
| Nr | Number of receive antennas | 1–4 |
| Modulation | Complex symbol constellation | QPSK, 16-QAM |
| Channel | Propagation model | AWGN, Rayleigh, Rician |
| SNR | Signal-to-noise ratio | Input –5 to 40 dB |
| Frames | Monte-Carlo frames per SNR | 1–2000 |
| Symbols/frame | Channel uses per frame | 10–1000 |
| LLL delta | Lovász parameter, LR only | 0.51–0.99 |

## Channel models

**AWGN:** a simple reference channel with additive complex Gaussian noise and an identity-like channel matrix in this implementation.
**Rayleigh:** independent complex Gaussian channel coefficients, representing rich scattering without a dominant line-of-sight component.
**Rician:** a deterministic line-of-sight component plus random scattering; the implementation uses K = 5.

QPSK carries 2 bits per symbol; 16-QAM carries 4 bits per symbol. The constellation symbols are normalised in the source code.

# 2.1 Maximum-Likelihood (ML)

*Key expression: x■ = arg min over x in S^Nt of ||y – Hx||²*

## How it works

The receiver evaluates every possible transmitted symbol vector and chooses the one that minimises the Euclidean distance between the received vector and the candidate's predicted vector.

## Why this detector?

ML is included as a high-performance reference. It makes a joint decision across all streams instead of detecting them independently.

## When to use it

Use for small antenna dimensions and small constellations, offline benchmarking, or when computation is less constrained.

## What is special?

Its defining feature is exhaustive joint search. With M constellation points and Nt streams, the candidate count is M^Nt.

## Trade-off and why not always use it

The search grows exponentially with Nt and modulation order. The program limits the candidate count to 100,000.

# 2.2 Zero-Forcing (ZF)

*Key expression: x■ = H†y; H† = (H■H)■¹H■ (full-column-rank case)*

## How it works

ZF applies the channel pseudo-inverse to separate the spatial streams, then slices the equalised values to the nearest constellation symbols.

## Why this detector?

It is a simple linear baseline that attempts to cancel inter-stream interference.

## When to use it

Use when the channel is well-conditioned, SNR is relatively high and low receiver complexity is important.

## What is special?

In an ideal noiseless full-rank system, channel inversion separates the streams.

## Trade-off and why not always use it

ZF does not account for noise variance. Weak channel directions can cause strong noise enhancement. This implementation requires Nr ≥ Nt and a full-column-rank channel.

# 2.3 Minimum Mean-Square Error (MMSE)

*Key expression: W = (H■H + (σn²/Es)I)■¹H■; x■ = Wy*

## How it works

MMSE uses a noise-dependent regularisation term while calculating the linear filter. This avoids aggressively inverting weak channel directions.

## Why this detector?

It balances residual inter-stream interference against noise amplification and is often a more robust linear detector than ZF in noisy conditions.

## When to use it

Use in low-to-moderate SNR or when channel conditioning makes ZF noise enhancement problematic.

## What is special?

It explicitly uses the noise variance. As noise becomes small, MMSE approaches ZF behaviour.

## Trade-off and why not always use it

It remains a linear detector and does not perform joint exhaustive search or successive cancellation.

# 2.4 Zero-Forcing Successive Interference Cancellation (ZF-SIC)

*Key expression: yres,new = yres,old – hk x■k*

## How it works

The receiver detects a stream, slices the estimate, reconstructs its received contribution and subtracts that contribution from the residual. It repeats the process for the remaining streams.

## Why this detector?

Cancellation reduces the interference left for later detections and can improve on one-shot ZF.

## When to use it

Use when sequential processing is acceptable and a better result than basic linear ZF is desired.

## What is special?

Its distinguishing feature is stream-by-stream cancellation. This implementation uses an adaptive ordering metric based on equaliser-row noise gain.

## Trade-off and why not always use it

A wrong early decision can be propagated to later stages. It requires repeated filtering and cancellation.

# 2.5 MMSE Successive Interference Cancellation (MMSE-SIC)

*Key expression: MMSE filtering + symbol slicing + residual cancellation*

## How it works

At each stage, an MMSE filter is calculated for the remaining channel columns. The selected stream is detected and its contribution is removed from the residual.

## Why this detector?

It combines noise-aware regularisation with the interference reduction of SIC.

## When to use it

Use when BER performance matters and the receiver can tolerate sequential processing.

## What is special?

It combines two useful mechanisms: MMSE limits noise enhancement; SIC removes already detected streams.

## Trade-off and why not always use it

It is more complex than one-shot MMSE and can still suffer error propagation after a wrong hard decision.

# 2.6 Lattice-Reduction-Aided (LR)

*Key expression: Hred = H T*

## How it works

Complex LLL-style reduction applies Gaussian-integer column operations to obtain a different, potentially better-conditioned channel basis. The implementation then uses MMSE-aided filtering and maps the estimate back before slicing.

## Why this detector?

It is included to study whether basis conditioning can help a linear detector when the original channel matrix is poorly conditioned.

## When to use it

Use when channel conditioning is a concern and preprocessing overhead is acceptable.

## What is special?

It changes the mathematical basis used for detection, not the physical propagation channel.

## Trade-off and why not always use it

Reduction adds computation. The Gaussian-integer transformation does not generally preserve a finite QPSK/16-QAM alphabet; therefore this implementation is educational LR-aided detection, not exact finite-alphabet ML. BER improvement is not guaranteed.

# 3. Implementation and execution

The repository provides twelve standalone source files: six Python programs and six C99 programs. Each program has an interactive console interface and prints a BER table and an example transmitted, received and detected vector. The Python programs use the Python standard library; no third-party Python package is required to run them.

## Python execution

From the repository root:

```
python src/python/ML_MIMO_Detection.py
```

Choose the detector file required. The program prompts for Nt, Nr, modulation, channel, selected SNR, frames and symbols per frame. LR also asks for the LLL delta.

## C99 execution

Example using GCC:

```
gcc src/c/ML_MIMO_Detection.c -o ml_mimo -std=c99 -lm
./ml_mimo
```

On Windows, execute `.\ml_mimo.exe`. Repeat with the corresponding source file for each detector.

## Reference run configuration

Nt = 2, Nr = 2, QPSK, Rayleigh fading, 100 frames, 50 symbols per frame and SNR points 0, 5, 10, 15, 20, 25 and 30 dB. For LR, delta = 0.75. Each SNR point contains 5,000 channel-use vectors, or 20,000 transmitted bits for two QPSK streams.

## BER definition

BER = erroneous detected bits / total transmitted bits. The estimate is calculated independently at each SNR point. More frames and symbols generally make the estimate more stable, particularly at high SNR where errors are rare.

## Input detail

The selected SNR input is accepted from –5 to 40 dB. The normal sweep remains 0–30 dB in 5 dB steps; a selected value outside that range is appended as an extra point. If the selected value is already within the sweep, the sweep is unchanged. The example vectors printed at the end correspond to the final SNR point.

# 4. Simulation results

The following reference data were generated by running all six Python programs with the same entered configuration. The corresponding raw console outputs and CSV data are included in the repository's results directory.

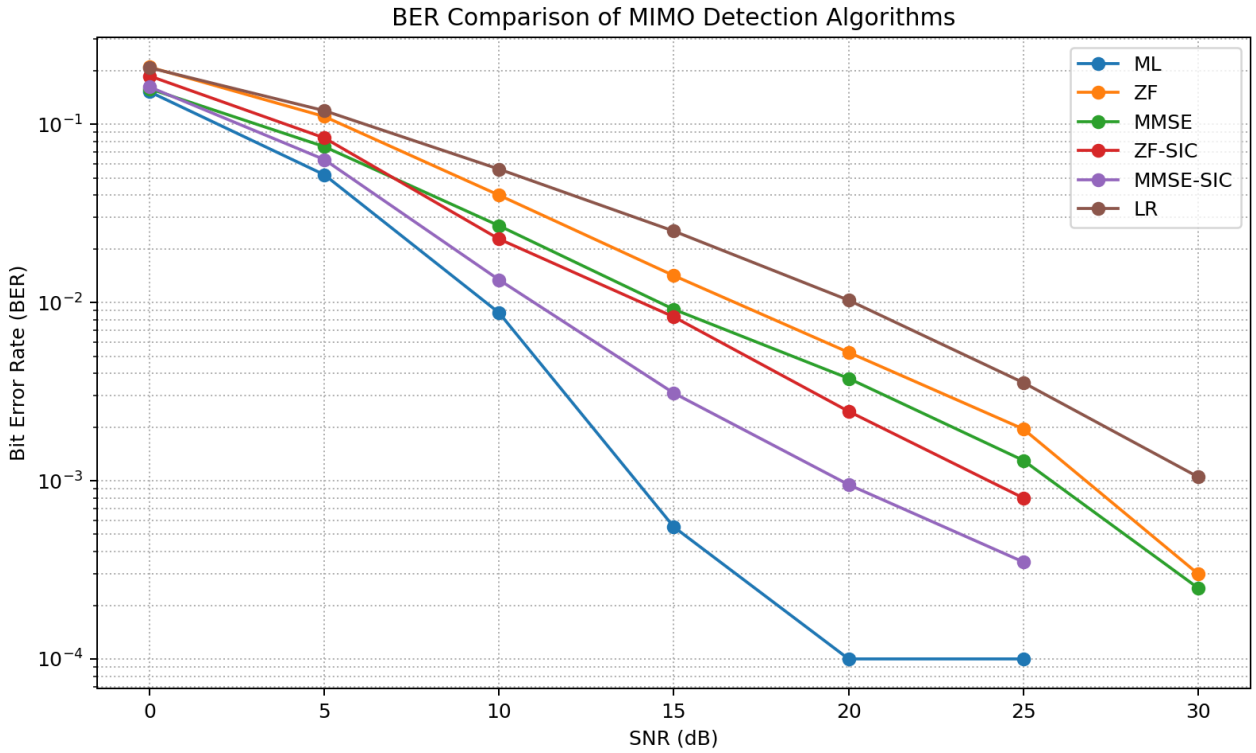



Figure 1. BER versus SNR for the 2 × 2 QPSK Rayleigh reference run. A logarithmic vertical axis is used; a zero measured BER point cannot be displayed on a logarithmic scale.

# 4.1 BER comparison table

| SNR (dB) | ML | ZF | MMSE | ZF-SIC | MMSE-SIC | LR |
|---|---|---|---|---|---|---|
| 0 | 0.15220 | 0.20940 | 0.15820 | 0.18665 | 0.16195 | 0.20760 |
| 5 | 0.05210 | 0.11055 | 0.07490 | 0.08385 | 0.06340 | 0.11925 |
| 10 | 0.00875 | 0.04000 | 0.02695 | 0.02270 | 0.01340 | 0.05600 |
| 15 | 0.00055 | 0.01415 | 0.00915 | 0.00830 | 0.00310 | 0.02515 |
| 20 | 0.00010 | 0.00525 | 0.00375 | 0.00245 | 0.00095 | 0.01030 |
| 25 | 0.00010 | 0.00195 | 0.00130 | 0.00080 | 0.00035 | 0.00355 |
| 30 | 0.00000 | 0.00030 | 0.00025 | 0.00000 | 0.00000 | 0.00105 |

## 4.2 Discussion

• BER falls as SNR increases for all six detectors in this particular run.
• ML has the lowest or tied-lowest measured BER at the tested SNR points, consistent with its role as a joint exhaustive detector under the assumed model.
• MMSE-SIC improves over one-shot MMSE at the higher SNR points in this run.
• ZF has higher BER than MMSE in this run, illustrating the effect of noise enhancement.
• The LR curve is worse in this run. The result should not be interpreted as proof that lattice reduction is generally inferior; the finite-alphabet mapping limitation and implementation details matter.

### Important statistical note

A measured BER of 0.00000 means that no bit errors were observed in the finite sample at that SNR. It does not mean the theoretical probability of error is exactly zero. For stronger claims, increase the number of bits and repeat the experiment with several independent seeds.

# 5. Limitations and future improvements

## Current scope and limitations

• The simulation is baseband and flat-fading; it does not model a full OFDM or cellular/WLAN physical layer.
• Channel state information is assumed known at the receiver; channel-estimation errors are not modelled.
• Synchronisation errors, carrier-frequency offset, phase noise, coding, interleaving, RF impairments and frequency-selective multipath are outside the current scope.
• Monte-Carlo BER depends on the sample count and random realisations.
• ML becomes expensive as the number of streams and constellation size increase.
• SIC methods can experience error propagation.
• The LR implementation's finite-constellation transformation requires careful interpretation.

## Recommended future work

1. Add automated batch runs and save configuration metadata with every result.
2. Run multiple random seeds and calculate confidence intervals.
3. Add theoretical or reference BER curves where an appropriate analytical expression is available.
4. Compare runtime and operation counts in addition to BER.
5. Add channel-estimation error and frequency-selective/OFDM scenarios.
6. Validate numerical agreement between the Python and C implementations using identical test vectors.
7. Investigate finite-alphabet-aware LR detection or a well-established LR-aided receiver formulation.

## Conclusion

The project provides a practical side-by-side learning framework for six widely studied MIMO detection approaches. ZF and MMSE establish linear baselines; SIC methods add sequential interference cancellation; ML provides an exhaustive joint-search reference; and LR introduces channel-basis preprocessing. The results illustrate that detector selection is a trade-off among BER, complexity, noise sensitivity and implementation constraints.

## Authors

**PRANTHI (123EC0048)**
**CHETHAN (123EC0049)**

## Repository contents

Source code: `src/python/` and `src/c/`
Theory: `docs/ALGORITHMS.md`
Methodology: `docs/SIMULATION_METHODOLOGY.md`
Data, plots and captured console output: `results/`